Auravo — AI Communication Coach MVP

Auravo is an AI-powered SaaS platform that helps students and professionals improve their English communication skills through personalized learning paths, simulated conversations, progress tracking with replay, and meeting preparation coaching. The MVP focuses on four core features that together create a compelling practice-feedback-growth loop, driving long-term retention while remaining affordable for price-sensitive users.

Goals

Business Goals

- Acquire 10,000 registered users within 3 months of launch, with at least 30% converting from free to paid tier.
- Achieve a monthly retention rate of 60% or higher by Month 3, demonstrating the stickiness of the adaptive learning loop.
- Reach $20K MRR within 6 months through a combination of micro-subscriptions and student pricing.
- Establish a Net Promoter Score (NPS) of 40+ within the first quarter post-launch.
- Build a foundation for word-of-mouth growth with a referral-driven sign-up rate of at least 15%.

User Goals

- Receive a personalized learning plan that adapts to individual weaknesses in real time, eliminating guesswork about what to practice.
- Practice speaking in realistic, judgment-free simulated scenarios at any time of day without scheduling a tutor.
- See measurable proof of improvement over time through recordings, scores, and visual progress comparisons.
- Prepare for high-stakes speaking moments (meetings, presentations, interviews) with targeted AI coaching.
- Build lasting communication confidence through consistent, structured daily practice.

Non-Goals

- Building a live human-tutoring marketplace or matching users with human coaches.
- Supporting languages other than English in the MVP.
- Creating a social network or community features (leaderboards, peer matching) in the first release.

User Stories

Student (Undergraduate or Graduate)

- As a student, I want to take an initial assessment so that the platform understands my current English speaking level and builds a plan around my weaknesses.
- As a student, I want to practice mock interview conversations so that I can prepare for campus placement and internship interviews.
- As a student, I want to replay my early recordings alongside recent ones so that I can hear and see how much I have improved over the semester.
- As a student, I want my learning path to adjust automatically when I improve in one area so that I am always working on the right skill.

Working Professional

- As a professional, I want to simulate a client call or team standup so that I can practice professional communication in a safe environment.
- As a professional, I want to paste my upcoming meeting agenda and rehearse talking points so that I walk into the meeting feeling prepared and confident.
- As a professional, I want a weekly progress report showing my improvements in filler words, pacing, and vocabulary so that I can track ROI on my practice time.
- As a professional, I want the AI to give me feedback on my tone and clarity after each simulation so that I know exactly what to improve next.

Career Switcher or Job Seeker

- As a job seeker, I want to practice behavioral interview questions with AI follow-ups so that I can build confidence for real interviews.
- As a job seeker, I want the platform to identify my weakest communication areas and prioritize them so that I make the most progress in limited time before my interview date.
- As a job seeker, I want to set a goal date for my interview and receive a backward-planned practice schedule so that I am fully prepared by that date.

Functional Requirements

Adaptive Learning Paths (Priority: P0 — Core)

- Initial Assessment Engine: A 5–7 minute spoken assessment at onboarding that evaluates pronunciation, grammar, fluency, vocabulary range, filler word usage, and pacing. Produces a baseline score across each dimension.
- Personalized Curriculum Generator: AI generates a weekly learning plan with daily practice sessions (10–20 minutes each) tailored to the user's weakest areas.
- Real-Time Adaptation: After each session, the system re-evaluates the user's scores and adjusts upcoming sessions. If pronunciation improves but filler words persist, the next sessions shift focus accordingly.
- Difficulty Progression: Content difficulty scales automatically across beginner, intermediate, and advanced levels based on cumulative performance.

- Skill Dimension Tracking: Tracks at least six dimensions — pronunciation, grammar, fluency, vocabulary, filler words, and pacing — each with an independent score from 0 to 100.

Simulated Conversations (Priority: P0 — Core)

- Scenario Library: At least 30 pre-built scenarios at launch, spanning categories such as job interviews, client calls, team meetings, elevator pitches, casual networking, academic presentations, and customer service interactions.
- Dynamic AI Responses: The AI conversation partner responds contextually, asks follow-up questions, and adapts to the user's responses rather than following a rigid script.
- Real-Time Feedback Post-Session: After each simulation, the user receives a detailed scorecard covering grammar, vocabulary usage, filler words, confidence tone, clarity, and relevance of responses.
- Difficulty Levels per Scenario: Each scenario has easy, medium, and hard variants (e.g., a friendly interviewer vs. a challenging one).
- Custom Scenario Creation: Users can describe a scenario in free text and the AI generates a conversation simulation around it.

Progress Journal and Replay (Priority: P0 — Core)

- Session Recording: Every speaking session is recorded and stored securely, accessible to the user at any time.
- Timeline View: A chronological timeline showing all sessions with date, scenario type, duration, and overall score.
- Side-by-Side Replay: Users can select any two recordings and play them back-to-back or in split view with score comparisons displayed alongside.
- Progress Dashboard: Visual charts showing improvement trends across each skill dimension over weeks and months.
- Milestone Markers: Automatic markers on the timeline when significant improvements are detected (e.g., "Filler words reduced by 50%" or "Vocabulary score crossed 70").
- Exportable Progress Report: Users can download or share a PDF summary of their progress for personal records or to show employers and mentors.

Meeting Prep Mode (Priority: P0 — Core)

- Agenda Input: Users paste a meeting agenda, presentation topic, or talking points into a free-text field.
- Context Configuration: Users specify meeting type (presentation, negotiation, standup, 1-on-1), audience (peers, leadership, clients), and duration.
- AI-Generated Practice Simulation: Based on the input, the AI creates a realistic simulation including likely questions, pushback scenarios, and audience reactions.
- Talking Point Coach: Before the simulation, the AI suggests structured talking points, transitions, and strong opening and closing statements.
- Post-Rehearsal Feedback: Detailed feedback on pacing, clarity, persuasiveness, filler words, and alignment between stated agenda and what was actually communicated.
- Quick Prep Option: A 5-minute rapid-fire mode for users who need last-minute preparation before an imminent meeting.

User Account and Onboarding (Priority: P0 — Supporting)

- Sign-Up and Authentication: Email and social login (Google, Apple). Student verification via .edu email for discounted pricing.
- Onboarding Flow: Guided 3-step onboarding — select goal (interview prep, professional growth, general improvement), take initial assessment, receive first learning plan.
- Profile and Preferences: Users set preferred practice time, session duration, target accent (American, British, neutral), and areas of focus.
- Notification and Reminders: Configurable daily reminders to practice, streak notifications, and weekly progress email summaries.

User Experience

Entry Point and First-Time User Experience

- Users discover Auravo through app store listings, web search, social media, or referral links.
- Landing page communicates the core value proposition: "Improve your English speaking with a personal AI coach Voca — practice anytime, track your growth, prepare for real moments."
- Sign-up requires only email or social login. Student users can verify .edu email for discounted pricing during or after sign-up.
- Immediately after sign-up, users enter a 3-step onboarding flow.

Core Experience

- Step 1: Goal Selection
    - User selects a primary goal from options: Interview Preparation, Professional Communication, Academic Speaking, or General Improvement.
    - UI displays a brief description of what each path focuses on.
    - Selection is stored but can be changed later from settings.

- Step 2: Initial Assessment
    - User is prompted to complete a 5–7 minute spoken assessment.
    - The assessment includes reading a passage aloud, answering two open-ended questions, and describing a visual prompt.
    - Microphone permission is requested with a clear explanation of why audio access is needed.
    - A progress bar shows assessment completion. If the user drops off, they can resume later.
    - On completion, the AI processes the recording and generates baseline scores across six skill dimensions within 15 seconds.

- Step 3: Personalized Dashboard
    - User lands on their home dashboard showing their skill radar chart (six dimensions), today's recommended session, and current streak count.

    - The recommended session is auto-generated based on the assessment results.
    - A "Start Practice" button launches the daily session. A "Meeting Prep" button is prominently visible for professionals.

- Step 4: Daily Practice Session (Adaptive Learning Path)
    - Session begins with a brief context screen explaining today's focus area (e.g., "Today we are working on reducing filler words during explanations").
    - User completes 2–3 speaking exercises within 10–20 minutes.
    - After each exercise, real-time feedback appears: a transcript with highlighted issues, scores per dimension, and a brief AI-written coaching note.
    - At session end, a summary card shows scores, comparison to previous session, and a motivational nudge.

- Step 5: Simulated Conversation
    - User navigates to the Simulations tab and browses scenarios by category or searches by keyword.
    - User selects a scenario and difficulty level, then taps "Start Simulation."
    - The AI introduces the scenario context (e.g., "You are in a final-round interview for a product manager role. I am the hiring manager. Let us begin.").
    - The conversation proceeds turn-by-turn. The user speaks, the AI responds naturally, and the exchange continues for 3–10 minutes depending on the scenario.
    - After the simulation ends, a detailed scorecard appears with playback of the full conversation, annotated transcript, and dimension-level scores.

- Step 6: Progress Review
    - User navigates to the Progress tab to see their timeline and charts.
    - They select two recordings for side-by-side comparison. Audio plays simultaneously or sequentially with visual score overlays.
    - Milestone badges are visible on the timeline (e.g., "First 7-Day Streak," "Grammar Score Hit 80").
    - User can export a progress report as a PDF.

- Step 7: Meeting Prep
    - User navigates to Meeting Prep and pastes their agenda or topic.
    - They configure context: meeting type, audience, and duration.
    - AI generates suggested talking points. User reviews and optionally edits them.
    - User taps "Start Rehearsal" and enters a simulated version of the meeting.
    - Post-rehearsal feedback covers pacing, clarity, persuasiveness, agenda alignment, and specific improvement suggestions.
    - User can repeat the rehearsal or save the feedback for reference before the actual meeting.

Advanced Features and Edge Cases

- If a user's microphone fails or audio quality is too low, the system detects this within 5 seconds and prompts the user to check their setup before continuing.
- If the initial assessment is interrupted, progress is saved and the user can resume from where they left off.

- If a user's skill scores plateau for more than two weeks, the system triggers a "plateau breaker" — a new type of exercise or challenge to re-engage them.
- Sessions that are too short (under 30 seconds of speech) are flagged and not counted toward progress to prevent gaming.
- Recordings are retained for 12 months on the free tier and indefinitely on paid plans. Users are notified before any deletion.

UI/UX Highlights

- Voice-first interface: the primary interaction is speaking, not typing. All instructions are also available as audio.
- Minimal screen clutter during speaking exercises — a simple waveform animation, a timer, and a stop button. Feedback appears only after the user finishes.
- Radar chart for skill dimensions uses distinct, colorblind-accessible colors.
- Mobile-first responsive design. The product must work well on smartphones since many students will use it on the go.
- Dark mode support from launch.
- Onboarding can be completed in under 3 minutes (excluding the assessment itself).
- All AI-generated feedback uses plain, encouraging language at an accessible reading level.

Narrative

Priya is a final-year engineering student in Bangalore. She has strong technical skills, but every time she enters a placement interview, her confidence drops. She stumbles over filler words, her pacing is uneven, and she struggles to articulate her project experience clearly. Tutoring is expensive and scheduling is inconvenient, so she has been practicing alone with no feedback.

She discovers Auravo through a college WhatsApp group. She signs up in under a minute using her Google account, selects "Interview Preparation" as her goal, and completes the initial assessment on her phone during her commute. The AI identifies her top weaknesses: filler words and response structure. Her first personalized session focuses on answering "Tell me about yourself" with a clear framework.

Over the next four weeks, Priya practices 15 minutes a day. She runs through mock interview simulations with increasingly tough AI interviewers. Each week, she checks her Progress Journal and sees her filler word count dropping and her clarity score climbing. The side-by-side replay of her Week 1 vs. Week 4 recordings is striking — she sounds like a different person.

The day before her interview at a top tech company, she uses Meeting Prep mode. She pastes the job description and role details, and the AI generates a realistic final-round simulation. She rehearses twice, refines her answers based on feedback, and walks into the interview room the next day feeling prepared. She gets the offer. Auravo becomes the tool she recommends to every classmate.

Success Metrics

User-Centric Metrics

- Day 7 retention rate: target 50% of users who complete onboarding return on Day 7.
- Day 30 retention rate: target 35% of onboarded users are still active at Day 30.
- Average sessions per week per active user: target 4 or more.
- Assessment completion rate: target 75% of users who start the assessment finish it.
- User satisfaction score (in-app survey): target 4.2 out of 5 or higher.

Business Metrics

- Free-to-paid conversion rate: target 30% within 60 days of sign-up.
- Monthly Recurring Revenue: target $20K MRR by Month 6.
- Customer Acquisition Cost: target under $5 per registered user via organic and referral channels.
- Churn rate for paid subscribers: target below 8% monthly.

Technical Metrics

- Speech-to-text processing latency: under 3 seconds for a 2-minute recording.
- AI feedback generation latency: under 5 seconds post-session.
- Platform uptime: 99.5% or higher.
- App crash rate: below 1% of sessions.

Tracking Plan

- User signs up (method: email, Google, Apple)
- Onboarding goal selected (goal type)
- Assessment started, completed, or abandoned (step reached)
- Daily session started, completed (duration, focus area)
- Simulation started, completed (scenario category, difficulty, score)
- Progress tab viewed (frequency, recordings replayed)
- Side-by-side replay used (recordings compared)
- Meeting Prep session started, completed (meeting type, rehearsal count)
- PDF progress report exported
- Subscription started, upgraded, or cancelled (plan type, tenure)
- Daily streak count updated
- Push notification opened (notification type)

Technical Considerations

Recommended Tech Stack (Cost-Optimized for MVP)

Frontend

- React.js with Next.js for the web app (free, open-source, great SEO, server-side rendering)
- React Native for mobile apps to share code with web, reducing dev cost
- Tailwind CSS for rapid UI development without a designer
- Web Audio API and MediaRecorder API for browser-based audio capture
- Estimated cost: $0 (all open-source)

Backend

- Node.js with Express.js or Fastify for the API server (lightweight, fast, free)
- PostgreSQL as the primary relational database (free, robust, handles user data, scores, session metadata)
- Redis for session caching, rate limiting, and real-time streak tracking
- Bull or BullMQ for job queues to handle async audio processing and AI inference
- Estimated cost: $0 for software, hosting costs only

AI and Speech Processing

- OpenAI Whisper (open-source, self-hosted) as the primary speech-to-text engine. Run the medium or large-v3 model on GPU instances for high accuracy across Indian, American, and British accents. Cost: approximately $0.10–0.20 per hour of audio on a self-hosted GPU vs $0.006/min on Whisper API
- Whisper.cpp for on-device or edge inference to reduce server costs for shorter recordings
- OpenAI GPT-4o-mini for conversation simulation and feedback generation. At approximately $0.15 per 1M input tokens and $0.60 per 1M output tokens, this is affordable for multi-turn conversations
- Fallback: Llama 3.1 8B or Mistral 7B (self-hosted, open-source) for cost-free feedback generation if API costs need further reduction
- Groq API as an alternative LLM inference provider offering near-instant responses at lower cost than OpenAI for supported open-source models
- Librosa (Python library, free) for audio feature extraction: pitch analysis, speech rate, pause detection, and energy levels for tone and pacing scoring
- spaCy or LanguageTool (both open-source) for grammar checking and sentence structure analysis on transcripts
- Estimated AI cost per user session (10 min): approximately $0.02–0.05

Audio Storage and Processing

- AWS S3 or Cloudflare R2 for audio file storage. R2 has zero egress fees, saving significantly on playback costs. Estimated cost: $0.015/GB/month on S3, $0 egress on R2
- FFmpeg (open-source) for server-side audio transcoding and compression to Opus format, reducing file sizes by ~80% vs WAV
- Audio recordings compressed to ~24kbps Opus, yielding roughly 1MB per 5 minutes. At 10,000 active users doing 4 sessions per week, total storage is approximately 160GB/month or roughly $2.40/month on R2

Infrastructure and Hosting

- Railway or Render for backend deployment during MVP phase (simple, affordable, starts at $5/month per service)
- Alternative: AWS EC2 Spot Instances or Hetzner Cloud for GPU workloads (Whisper inference). Hetzner offers GPU servers starting at approximately $1.30/hour vs $3+/hour on AWS
- Vercel for frontend hosting (free tier covers MVP traffic easily)
- Cloudflare for CDN, DDoS protection, and DNS (free tier)
- Supabase as an alternative to self-managed PostgreSQL (free tier includes 500MB database, auth, and real-time subscriptions)
- Estimated monthly infrastructure cost for MVP (up to 10,000 users): $150–$400/month

Authentication and Payments

- NextAuth.js or Supabase Auth for authentication (Google, Apple, email login) — free
- Stripe for international payments (2.9% + $0.30 per transaction, no monthly fee)
- Razorpay for Indian market payments (2% per transaction, supports UPI which is critical for student affordability)
- No upfront licensing costs for either payment provider

Analytics, Monitoring, and Notifications

- PostHog (open-source, self-hosted or cloud free tier up to 1M events/month) for product analytics to save costs
- Sentry (free tier: 5K errors/month) for error tracking
- Uptime Robot (free tier: 50 monitors) for uptime monitoring
- Firebase Cloud Messaging (free) for push notifications
- Resend (free tier: 3,000 emails/month) for transactional emails and weekly progress reports

Cost Summary Table

Component	Tool/Service	Monthly Cost (MVP Scale)
Frontend Hosting	Vercel Free Tier	$0
Backend Hosting	Railway or Render	$20–50
Database	Supabase Free Tier or PostgreSQL on Railway	$0–25
GPU for Whisper	Hetzner Cloud GPU (on-demand)	$50–150
LLM API (GPT-4o-mini)	OpenAI API	$40–100
Audio Storage	Cloudflare R2	$3–10
CDN and Security	Cloudflare Free	$0

Email	Resend Free Tier	$0
Analytics	PostHog Free Tier	$0
Payments	Stripe/Razorpay	Transaction fees only
Push Notifications	Firebase FCM	$0
Monitoring	Sentry + Uptime Robot Free	$0
Total Estimated Monthly Cost for up to 10,000 users		$113–335/month

Cost Optimization Strategy

- Self-host Whisper on GPU spot instances during off-peak hours and use Whisper API as fallback during peak to balance cost and latency
- Use GPT-4o-mini for all LLM tasks initially. Migrate high-volume, template-based feedback to self-hosted Llama or Mistral models once patterns stabilize
- Cache common AI feedback patterns. Many users make similar mistakes — cache coaching responses for common filler word patterns, grammar errors, and vocabulary suggestions
- Implement progressive audio retention: keep only the last 90 days of recordings for free-tier users, compress older recordings further for paid users
- Use edge functions (Cloudflare Workers or Vercel Edge) for lightweight API calls to reduce backend server load
- Batch Whisper processing: queue recordings and process in batches on GPU instances rather than running instances continuously

---

Whisper Self-Hosting Setup Guide

Self-hosting OpenAI Whisper is the single largest cost-saving decision in the Auravo stack. This section provides a complete setup guide covering model selection, infrastructure, deployment architecture, API wrapper, performance tuning, and scaling strategy.

Model Selection and Comparison

Model	Parameters	VRAM Required	Relative Speed	English WER	Recommended Use
tiny	39M	~1GB	~32x real-time	~8%	Development and testing only
base	74M	~1GB	~16x real-time	~5.2%	Quick previews or edge devices
small	244M	~2GB	~6x real-time	~3.4%	Budget production with acceptable accuracy

medium	769M	~5GB	~2x real-time	~2.7%	Recommended for MVP — best cost-accuracy balance
large-v3	1550M	~10GB	~1x real-time	~2.0%	Premium tier or accent-heavy audio
large-v3-turbo	809M	~6GB	~3x real-time	~2.2%	Best option when available — near large-v3 accuracy at medium speed

Recommendation for Auravo: Start with the medium model for the MVP. It processes a 10-minute recording in approximately 5 minutes on a mid-tier GPU, with a word error rate low enough for reliable grammar and filler word analysis. Upgrade to large-v3-turbo for premium subscribers or when real-time transcription is needed.

Recommended GPU Infrastructure

Provider	GPU Option	VRAM	Hourly Cost	Monthly Cost (reserved)	Best For
Hetzner Cloud	NVIDIA A100 (partial)	40GB	~$1.30/hr	~$450/mo	Primary production workload
Hetzner Cloud	NVIDIA RTX 4000	16GB	~$0.60/hr	~$200/mo	MVP with medium model
RunPod	NVIDIA A40 (on-demand)	48GB	~$0.79/hr	Pay-as-you-go	Burst capacity and testing
RunPod	NVIDIA RTX 3090	24GB	~$0.44/hr	Pay-as-you-go	Cost-optimized batch processing
Vast.ai	Various (marketplace)	Varies	~$0.20–0.80/hr	Pay-as-you-go	Cheapest option for non-critical batch jobs
AWS EC2	g5.xlarge (A10G)	24GB	~$1.01/hr	~$450/mo	Fallback if other providers are unavailable

Recommendation for Auravo MVP: Use Hetzner RTX 4000 ($0.60/hr) as the primary GPU for the Whisper medium model. Use RunPod RTX 3090 ($0.44/hr) for overflow batch processing. Do not run GPU instances 24/7 at MVP scale — use on-demand instances triggered by job queues.

Docker Deployment Setup

Use a Docker-based deployment to simplify portability and reproducible environments.

```
FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04

RUN apt-get update && apt-get install -y python3 python3-pip ffmpeg && rm -rf /var/lib/apt/lists/*
```

```
RUN pip3 install openai-whisper faster-whisper torch torchaudio

# Download model at build time to avoid runtime downloads
RUN python3 -c "import whisper; whisper.load_model('medium')"

WORKDIR /app
COPY whisper_server.py .

EXPOSE 8000
CMD ["python3", "whisper_server.py"]
```

Docker Compose to orchestrate Whisper API and Redis queue:

```
version: "3.8"
services:
  whisper-api:
    build: .
    ports:
      - "8000:8000"
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]
    environment:
      - WHISPER_MODEL=medium
      - MAX_CONCURRENT=3
      - BEAM_SIZE=5
    volumes:
      - whisper-cache:/root/.cache/whisper
      - audio-uploads:/app/uploads
    restart: unless-stopped

  redis-queue:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data

volumes:
  whisper-cache:
  audio-uploads:
  redis-data:
```

API Wrapper Service

Auravo needs a lightweight FastAPI wrapper around Whisper that accepts audio uploads, queues processing, and returns transcription results with word-level timestamps. Use faster-whisper for inference.

```
from fastapi import FastAPI, UploadFile, BackgroundTasks
from faster_whisper import WhisperModel
import redis
import json
import uuid
```

```
import os

app = FastAPI()
model = WhisperModel("medium", device="cuda", compute_type="float16")
redis_client = redis.Redis(host="redis-queue", port=6379, db=0)

@app.post("/transcribe")
async def transcribe(file: UploadFile, background_tasks: BackgroundTasks):
    job_id = str(uuid.uuid4())
    file_path = f"/app/uploads/{job_id}.opus"

    with open(file_path, "wb") as f:
        content = await file.read()
        f.write(content)

    background_tasks.add_task(process_audio, job_id, file_path)
    return {"job_id": job_id, "status": "queued"}

def process_audio(job_id: str, file_path: str):
    segments, info = model.transcribe(
        file_path,
        beam_size=5,
        language="en",
        word_timestamps=True,
        vad_filter=True,
        vad_parameters=dict(min_silence_duration_ms=500)
    )

    result = {
        "job_id": job_id,
        "language": info.language,
        "duration": info.duration,
        "segments": []
    }

    for segment in segments:
        result["segments"].append({
            "start": segment.start,
            "end": segment.end,
            "text": segment.text,
            "words": [
                {"word": w.word, "start": w.start, "end": w.end,
"probability": w.probability}
                for w in segment.words
            ] if segment.words else []
        })

    redis_client.set(f"transcription:{job_id}", json.dumps(result), ex=3600)
    os.remove(file_path)

@app.get("/result/{job_id}")
async def get_result(job_id: str):
    result = redis_client.get(f"transcription:{job_id}")
    if result:
        return json.loads(result)
    return {"status": "processing"}
```

Note: faster-whisper is used instead of the standard openai-whisper library because it is 4x faster and uses 2x less memory through CTranslate2 optimizations. Word-level timestamps are critical for Auravo's filler word detection, pacing analysis, and pause measurement.

Faster-Whisper vs Standard Whisper

Feature	openai-whisper	faster-whisper
Speed	1x baseline	4x faster (CTranslate2)
Memory	Full model in VRAM	~50% less VRAM via int8/float16
Word timestamps	Supported	Supported (more reliable)
VAD filter	Not built-in (needs external)	Built-in (silero-vad)
Streaming support	No	Partial (segment-level)
Batch processing	Manual	Built-in batched inference
Recommendation	Use for testing only	Use for production

Recommendation: Always use faster-whisper in production. The VRAM savings alone mean Auravo can run the medium model on a 4GB GPU card, or run large-v3 on a 6GB card — halving infrastructure costs.

Performance Tuning for Auravo

- Enable VAD (Voice Activity Detection) filtering to skip silence, reducing processing time by 20-40% on typical user recordings that include pauses and thinking time.
- Use float16 compute type on NVIDIA GPUs for 2x throughput vs float32 with negligible accuracy loss.
- Set beam_size to 5 for production quality. Use beam_size 1 for the Quick Prep mode where speed matters more than perfect accuracy.
- Pre-process audio with FFmpeg before sending to Whisper: convert to 16kHz mono WAV, apply noise reduction with the afftdn filter, and normalize volume. This improves accuracy for mobile recordings with background noise.

```
ffmpeg -i input.opus -ar 16000 -ac 1 -af "afftdn=nf=-25,loudnorm" output.wav
```

- Set max_concurrent workers based on GPU VRAM. For medium model on 16GB VRAM: max 3 concurrent jobs. For large-v3 on 24GB VRAM: max 2 concurrent jobs.
- Implement request prioritization in the Redis queue: real-time simulation transcription gets highest priority, daily practice sessions get normal priority, and batch re-processing for progress analytics gets lowest priority.

Scaling Architecture

Three-tier scaling approach:

Tier 1 — MVP (0 to 10,000 users): Single Hetzner GPU instance running faster-whisper medium model. On-demand GPU only — spin up when job queue has items, auto-shutdown after 15 minutes of idle time. Use Whisper API as a hot fallback when the GPU instance is booting (takes 30-60 seconds). Estimated processing capacity: approximately 500 audio-minutes per hour. Estimated cost: $50-150/month based on actual usage.

Tier 2 — Growth (10,000 to 50,000 users): Two GPU instances with a load balancer. One always-on instance for real-time simulation transcription. One on-demand instance for batch processing of daily sessions. Upgrade to large-v3-turbo model for better accuracy. Add a Redis-based priority queue to separate real-time and batch workloads. Estimated cost: $400-600/month.

Tier 3 — Scale (50,000+ users): Kubernetes cluster with GPU node pools for auto-scaling. Multiple faster-whisper replicas behind NVIDIA Triton Inference Server for optimized GPU utilization. Consider fine-tuning Whisper on Indian English accent data (requires ~100 hours of labeled audio collected with user consent) to improve accuracy from ~2.7% WER to under 2% for the primary user demographic. Evaluate moving to Whisper distilled models (distil-whisper) for 5x speed improvement with minimal accuracy loss. Estimated cost: scales linearly at approximately $0.001 per audio-minute processed.

Monitoring and Reliability

- Track transcription latency per job (p50, p95, p99) — target p95 under 30 seconds for a 10-minute recording.
- Monitor GPU utilization — alert if sustained above 90% for more than 10 minutes (signals need for scaling).
- Track Word Error Rate by sampling 1% of transcriptions for human review weekly. Flag sessions where user reports "incorrect transcript."
- Set up automatic failover to Whisper API when self-hosted GPU is unavailable, unhealthy, or queue depth exceeds 50 jobs.
- Log per-session processing costs (GPU time multiplied by hourly rate) to track actual cost per user and identify optimization opportunities.
- Use Prometheus and Grafana (both free, open-source) for GPU monitoring dashboards.

Migration Path from API to Self-Hosted

1. Week 1-2: Use OpenAI Whisper API ($0.006/minute) for all transcription. No GPU infrastructure needed. Focus engineering time on product features. Estimated cost for 1,000 beta users: approximately $30-50/month.
2. Week 3-4: Deploy Docker-based faster-whisper on a single Hetzner GPU. Route 50% of traffic to self-hosted, 50% to API. Compare accuracy and latency between the two. Keep API as automatic fallback.
3. Week 5 onward (post-launch): Route 90% of traffic to self-hosted. Use API only as fallback during GPU downtime or traffic spikes. Monitor cost savings — expected 60-70% reduction vs API-only approach at 10,000 users.
4. Month 3+ (if growing rapidly): Evaluate multi-GPU setup and Kubernetes migration. Begin collecting accent-specific audio data for potential fine-tuning.

LLM Setup Guide — GPT-4o-mini and Open-Source Fallbacks

The LLM layer powers four critical functions in Auravo: conversation simulation, session feedback generation, adaptive curriculum planning, and meeting prep coaching. This guide covers the primary GPT-4o-mini integration, prompting strategy for each function, structured

output handling, cost analysis, open-source fallback deployment, and a hybrid routing architecture to minimize costs while maintaining quality.

Why GPT-4o-mini as the Primary LLM

GPT-4o-mini offers the best balance of cost, speed, and quality for Auravo's use cases. At $0.15 per 1M input tokens and $0.60 per 1M output tokens, it is approximately 60x cheaper than GPT-4o while delivering sufficient quality for coaching feedback and conversation simulation. Its 128K context window comfortably handles multi-turn conversation histories. Response latency averages 300-600ms for first token, which is fast enough for turn-by-turn simulated conversations.

Prompting Strategy by Feature

Conversation Simulation Prompts

Simulated conversations require the LLM to stay in character, respond naturally, ask follow-up questions, and adjust difficulty based on the scenario level. System prompt template:

```
You are a conversation simulation engine for an English communication
coaching app called Auravo.

ROLE: {scenario_role} (e.g., "Senior hiring manager at a Fortune 500 tech
company")
SCENARIO: {scenario_description}
DIFFICULTY: {difficulty_level} (easy / medium / hard)
USER PROFILE: {user_level} (beginner / intermediate / advanced)

BEHAVIORAL RULES:
- Stay fully in character throughout the conversation. Never break character
or acknowledge you are an AI.
- Respond naturally as the character would in the given scenario.
- Keep responses concise — 2-4 sentences maximum per turn, mimicking real
conversation pace.
- On EASY difficulty: be encouraging, give clear signals, ask straightforward
questions.
- On MEDIUM difficulty: introduce occasional follow-up questions, ask for
clarification, and mildly challenge weak answers.
- On HARD difficulty: push back on vague answers, ask probing follow-ups,
introduce unexpected topic shifts, and simulate real pressure.
- If the user gives a very short or unclear response, prompt them to
elaborate rather than moving on.
- After 8-12 exchanges, naturally wind down the conversation with a closing
statement appropriate to the scenario.

IMPORTANT: Respond ONLY as the character. Do not provide coaching feedback
during the conversation — feedback is generated separately after the session
ends.
```

Each user turn in the messages array contains the Whisper transcript of what they spoke, and the assistant responds in character. A conversation_metadata object is appended to the system prompt with turn count, elapsed time, and remaining turns to help the LLM pace the conversation naturally.

Session Feedback Generation Prompts

After each practice session or simulation, the LLM generates structured coaching feedback. This is the highest-volume LLM call in the system. System prompt template:

```
You are an expert English communication coach analyzing a student's spoken
English. Generate structured, actionable feedback based on the transcript and
audio analysis data provided.

USER LEVEL: {user_level}
SESSION TYPE: {session_type} (daily_practice / simulation / meeting_prep)
FOCUS AREA: {focus_area} (e.g., "filler words", "grammar", "vocabulary")

AUDIO ANALYSIS DATA (from Librosa/spaCy pipeline):
- Words per minute: {wpm}
- Filler word count: {filler_count}
- Filler words detected: {filler_list}
- Pause count (>2 seconds): {pause_count}
- Average sentence length: {avg_sentence_length}
- Grammar errors detected: {grammar_errors}
- Vocabulary diversity score: {vocab_score}

TRANSCRIPT:
{full_transcript}

Generate feedback in the following JSON structure. Be encouraging but honest.
Use plain language at a reading level accessible to non-native English
speakers. Limit each suggestion to 1-2 sentences. Provide exactly 3
improvement suggestions prioritized by impact.
```

Expected JSON output schema:

```
{
  "overall_score": 72,
  "dimension_scores": {
    "grammar": 68,
    "vocabulary": 75,
    "fluency": 70,
    "pronunciation": 78,
    "filler_words": 55,
    "pacing": 80
  },
  "coaching_summary": "You communicated your ideas clearly and maintained
good pacing throughout. Your main area for improvement is reducing filler
words, which appeared 14 times in this session.",
  "top_suggestions": [
    {
      "area": "filler_words",
      "suggestion": "Try pausing silently for 1-2 seconds instead of saying
'um' or 'like'. Practice by recording yourself answering a question and
consciously replacing fillers with pauses.",
      "priority": 1
    },
    {
      "area": "grammar",
      "suggestion": "Watch your subject-verb agreement with plural nouns. You
said 'the team are' twice — in American English, use 'the team is' since team
is treated as singular.",
      "priority": 2
```

```
    },
    {
      "area": "vocabulary",
      "suggestion": "You used 'good' and 'nice' frequently. Try substituting
with more specific words like 'effective', 'compelling', or 'well-structured'
to sound more professional.",
      "priority": 3
    }
  ],
  "motivational_note": "Your fluency has noticeably improved compared to your
last session. Keep practicing daily — you are making real progress.",
  "comparison_to_previous": {
    "improved": ["pacing", "fluency"],
    "declined": [],
    "unchanged": ["grammar", "vocabulary", "filler_words", "pronunciation"]
  }
}
```

Use structured JSON output with response_format: { type: "json_object" } in the OpenAI API call to ensure consistent, parseable responses that the frontend can render directly without additional processing.

Curriculum Generation Prompts

The curriculum engine calls the LLM once per week per active user (or after each session if real-time adaptation is enabled) to generate or adjust the learning plan. System prompt template:

```
You are a curriculum planning engine for an English communication coaching
platform. Based on the user's current skill scores, recent session history,
and stated goals, generate an optimized weekly learning plan.

USER PROFILE:
- Level: {user_level}
- Goal: {user_goal} (e.g., "interview_preparation")
- Goal deadline: {deadline_date} (if set, e.g., "2026-06-15")
- Weeks active: {weeks_on_platform}

CURRENT SKILL SCORES (0-100):
- Grammar: {grammar_score}
- Vocabulary: {vocabulary_score}
- Fluency: {fluency_score}
- Pronunciation: {pronunciation_score}
- Filler words: {filler_score}
- Pacing: {pacing_score}

RECENT TRENDS (last 2 weeks):
- Improving: {improving_dimensions}
- Declining: {declining_dimensions}
- Plateaued: {plateaued_dimensions}

SESSION HISTORY (last 7 sessions):
{session_summaries}

RULES:
- Generate exactly 5 daily sessions (Monday to Friday). Weekend sessions are
optional bonus.
- Each session should be 10-20 minutes.
- Allocate 60% of session time to the user's weakest or declining dimensions.
```

```
- Allocate 20% to maintaining improving dimensions.
- Allocate 20% to introducing new challenge areas to prevent staleness.
- If a deadline is set, increase intensity and focus on the goal-relevant
skills.
- If a dimension has plateaued for 2+ weeks, introduce a new exercise type
for that dimension.
- Include at least 1 simulation session and 1 focused drill session per week.

Output a JSON plan with daily sessions including: day, duration_minutes,
focus_areas, exercise_types, difficulty_level, and a brief description.
```

Meeting Prep Coaching Prompts

Meeting prep requires the LLM to first generate talking point suggestions and then run a context-aware simulation. Talking points generation prompt:

```
You are a communication coach helping a professional prepare for an upcoming
meeting.

MEETING CONTEXT:
- Type: {meeting_type} (presentation / negotiation / standup / 1-on-1 /
client call)
- Audience: {audience} (peers / leadership / clients / mixed)
- Duration: {meeting_duration}
- Agenda/Topic: {user_agenda_input}
- User's communication level: {user_level}

Generate structured talking points including:
1. A strong opening statement (1-2 sentences)
2. 3-5 key talking points with transition phrases between them
3. Anticipated questions the audience might ask, with suggested response
frameworks
4. A strong closing statement
5. Words and phrases to avoid, and better alternatives

For a {meeting_type} with {audience}, calibrate formality and technical depth
accordingly. If the meeting type is a negotiation, include persuasion
techniques. If it is a standup, keep everything concise and status-focused.
```

Token Budget Analysis

Feature	Avg Input Tokens	Avg Output Tokens	Cost per Call	Calls per User/Day	Daily Cost per Active User
Conversation Simulation (10 turns)	2,500	1,200	$0.001	1	$0.001
Session Feedback	1,800	800	$0.0008	1.5	$0.0012
Curriculum Generation	1,200	1,000	$0.0008	0.14 (weekly)	$0.0001
Meeting Prep (talking points + sim)	3,000	2,000	$0.0017	0.3	$0.0005
Total daily LLM cost per active user					~$0.003

Projected monthly LLM costs: approximately $30/month at 10,000 active users, approximately $150/month at 50,000 active users. These estimates assume GPT-4o-mini pricing as of early 2026. The per-user cost is low enough that LLM API costs are not a scaling bottleneck until well past 100,000 active users.

Prompt Engineering Best Practices for Auravo

- Always include user_level in prompts so the LLM calibrates language complexity and feedback depth. A beginner gets simpler vocabulary in feedback; an advanced user gets nuanced suggestions.
- Use structured JSON output (response_format: { type: "json_object" }) for all feedback and curriculum calls. This eliminates parsing errors and ensures the frontend can render responses reliably.
- Keep conversation simulation prompts under 500 tokens of system instructions. Long system prompts waste tokens on every turn of a multi-turn conversation.
- Implement prompt versioning in the database. Store each prompt template with a version number so you can A/B test prompt changes and roll back if feedback quality drops.
- Use temperature 0.3 for feedback and curriculum generation (consistency matters more than creativity). Use temperature 0.7 for conversation simulation (natural variation makes conversations feel realistic).
- Append a brief user history summary (3-5 sentences) to feedback prompts so the LLM can reference previous sessions. Example: "In your last session, you reduced filler words from 18 to 12. Let us try to get below 10 today."
- For conversation simulations, include a hidden turn-counter in the system prompt that tells the LLM how many turns remain. This prevents conversations from ending abruptly or dragging on too long.

Open-Source LLM Fallback Setup

When to Use Open-Source Models

- When GPT-4o-mini API experiences downtime or rate limiting
- For high-volume, template-based feedback where patterns are predictable (e.g., filler word coaching, basic grammar corrections)
- When the monthly API bill exceeds the cost of running self-hosted GPU inference
- For data privacy compliance in regions where sending user transcripts to OpenAI's API is a concern
- As a cost reduction strategy once the product reaches 50,000+ active users

Recommended Open-Source Models

Model	Parameters	VRAM Required	Quality vs GPT-4o-mini	Speed (tok/sec on A100)	Best For
Llama 3.1 8B Instruct	8B	~6GB (quantized)	~75-80%	~100	Session feedback, curriculum generation

Llama 3.1 70B Instruct	70B	~40GB (quantized)	~90-95%	~30	Conversation simulation (near GPT-4o-mini quality)
Mistral 7B Instruct v0.3	7B	~5GB (quantized)	~70-75%	~110	Lightweight feedback, grammar analysis
Phi-3 Medium (14B)	14B	~10GB (quantized)	~80-85%	~70	Good balance of quality and efficiency
Qwen2.5 7B Instruct	7B	~5GB (quantized)	~75-80%	~105	Strong multilingual understanding, accent-diverse users

Recommendation: Start with Llama 3.1 8B Instruct for feedback generation fallback. It handles structured JSON output well, runs on a single consumer GPU, and delivers acceptable coaching quality for common feedback patterns. Reserve Llama 3.1 70B for conversation simulation fallback where natural dialogue quality matters most.

Self-Hosted Deployment with Ollama

Ollama provides the simplest path to self-hosted LLM inference for development and small-scale production.

```
# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Pull the recommended models
ollama pull llama3.1:8b-instruct-q4_K_M
ollama pull mistral:7b-instruct-v0.3-q4_K_M

# Start the Ollama server (runs on port 11434 by default)
ollama serve

# Test with a sample request
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.1:8b-instruct-q4_K_M",
  "prompt": "Analyze this transcript for grammar errors: I goes to the store
yesterday and buyed some fruits.",
  "stream": false
}'
```

Note: q4_K_M quantization reduces model size by approximately 75% with minimal quality loss, making it possible to run 8B models on GPUs with just 6GB VRAM.

Production Deployment with vLLM

For production-grade self-hosted inference with higher throughput, use vLLM which supports continuous batching, PagedAttention, and OpenAI-compatible API endpoints.

```
pip install vllm

# Launch vLLM server with OpenAI-compatible API
```

```
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-8B-Instruct \
  --dtype float16 \
  --max-model-len 8192 \
  --gpu-memory-utilization 0.9 \
  --port 8001
```

Docker deployment for vLLM:

```
version: "3.8"
services:
  vllm-server:
    image: vllm/vllm-openai:latest
    ports:
      - "8001:8000"
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]
    environment:
      - MODEL_NAME=meta-llama/Llama-3.1-8B-Instruct
      - DTYPE=float16
      - MAX_MODEL_LEN=8192
      - GPU_MEMORY_UTILIZATION=0.9
    volumes:
      - model-cache:/root/.cache/huggingface
    restart: unless-stopped

volumes:
  model-cache:
```

The key advantage of vLLM is its OpenAI-compatible API — Auravo's backend code can switch between GPT-4o-mini and self-hosted models by simply changing the base URL and model name, with no other code changes needed.

Groq API as a Middle-Ground Option

Groq provides ultra-fast inference for open-source models on custom LPU hardware. It offers an OpenAI-compatible API, so integration is trivial. Pricing for Llama 3.1 8B on Groq is approximately $0.05 per 1M input tokens and $0.08 per 1M output tokens — roughly 3x cheaper than GPT-4o-mini. Latency is often under 100ms for first token, making it viable for real-time conversation simulation. The tradeoff is that Groq has usage limits on the free tier and the model quality is limited to the open-source model's capability. Recommendation: use Groq as the default for session feedback and curriculum generation, and GPT-4o-mini for conversation simulation where quality matters most.

Hybrid Routing Architecture

Auravo's backend should implement intelligent routing to send each LLM task to the most cost-effective provider that meets quality requirements.

```
class LLMRouter:
    def __init__(self):
```

```
        self.providers = {
            "gpt4o_mini": OpenAIProvider(model="gpt-4o-mini"),
            "groq_llama": GroqProvider(model="llama-3.1-8b-instant"),
            "self_hosted": VLLMProvider(base_url="http://vllm-server:8001"),
        }

    def route_request(self, task_type: str, priority: str) -> str:
        # Conversation simulation — highest quality needed
        if task_type == "conversation_simulation":
            return "gpt4o_mini"

        # Meeting prep — high quality, user-facing
        if task_type == "meeting_prep":
            return "gpt4o_mini"

        # Session feedback — high volume, structured output
        if task_type == "session_feedback":
            if priority == "real_time":
                return "groq_llama"  # Fast, cheap
            return "self_hosted"     # Cheapest for batch

        # Curriculum generation — weekly, not time-sensitive
        if task_type == "curriculum_generation":
            return "self_hosted"

        # Default fallback
        return "gpt4o_mini"

    def call_with_fallback(self, task_type, prompt, **kwargs):
        primary = self.route_request(task_type, kwargs.get("priority"))
        fallback_chain = ["groq_llama", "gpt4o_mini"]

        try:
            return self.providers[primary].complete(prompt, **kwargs)
        except Exception:
            for fallback in fallback_chain:
                try:
                    return self.providers[fallback].complete(prompt,
**kwargs)
                except Exception:
                    continue
            raise LLMUnavailableError("All LLM providers failed")
```

This routing architecture ensures that the most expensive provider (GPT-4o-mini) is only used where quality directly impacts user experience, while cheaper options handle high-volume background tasks. The fallback chain provides resilience — if any single provider goes down, traffic automatically routes to the next available option.

Response Caching Strategy

Many Auravo users make similar mistakes and need similar coaching. Implement a semantic caching layer to avoid redundant LLM calls:

- Cache feedback for common filler word patterns. If a user's filler word list matches a previously seen pattern (e.g., "um", "like", "you know" with count between 10-15), serve the cached coaching suggestion instead of calling the LLM.
- Cache curriculum templates by user-level and goal-type combinations. There are approximately 12 combinations (3 levels times 4 goals). Pre-generate baseline

curriculum plans for each combination and only call the LLM for personalized adjustments.
- Cache conversation simulation opening messages by scenario. The first 2 turns of each scenario are largely identical regardless of user — pre-generate and cache these.
- Use Redis with a 24-hour TTL for feedback cache entries. Use PostgreSQL for longer-lived curriculum template cache.
- Estimated cache hit rate after 1 month of operation: 30-40% of feedback calls, reducing LLM API costs by an additional 30%.

Quality Assurance for LLM Outputs

- Implement an automated quality check on every LLM response: validate JSON structure, check that scores are within 0-100 range, verify that the number of suggestions matches the requested count, and confirm that the response language is English.
- Run a weekly "LLM-as-judge" evaluation: sample 50 feedback responses, send them to GPT-4o (the full model, not mini) with a rubric asking it to rate accuracy, helpfulness, and tone on a 1-5 scale. Track average scores over time and alert if quality drops below 3.5.
- Allow users to rate feedback with a simple thumbs-up/thumbs-down. Track the thumbs-down rate per LLM provider and per prompt version. If a provider's thumbs-down rate exceeds 15%, automatically route traffic away from it.
- Maintain a human-curated "golden set" of 100 transcript-feedback pairs that represent ideal coaching quality. Run new prompt versions against this golden set before deploying to production. Require 90%+ match rate on scoring accuracy (within 5 points of human-assigned scores).

Cost Scaling Projections

User Scale	Primary LLM	Fallback LLM	Est. Monthly LLM Cost	Cost per Active User
1,000 (beta)	GPT-4o-mini for all tasks	None needed	~$3/month	$0.003
10,000 (MVP)	GPT-4o-mini for sim + meeting prep; Groq for feedback	Self-hosted Llama 8B for curriculum	~$40/month	$0.004
50,000	GPT-4o-mini for sim only; Groq for meeting prep + feedback	Self-hosted Llama 8B + 70B	~$120/month	$0.0024
100,000	GPT-4o-mini for sim only; self-hosted for all else	Groq as backup	~$180/month	$0.0018
500,000	Self-hosted Llama 70B for sim; 8B for all else	GPT-4o-mini as premium fallback	~$600/month (GPU)	$0.0012

Per-user LLM cost decreases as scale increases because self-hosted GPU utilization improves and caching hit rates increase. LLM costs remain a small fraction of total operating costs at every scale.

Migration Playbook — API to Open-Source

- Phase 1 (Months 1-2, MVP launch): Use GPT-4o-mini for 100% of LLM calls. Simple, fast to implement, no infrastructure overhead. Total cost is negligible at beta scale.
- Phase 2 (Month 3, post-launch): Introduce Groq API for session feedback calls. Run both providers in parallel for 2 weeks, comparing quality scores and latency. If Groq quality is within 10% of GPT-4o-mini on the golden set evaluation, make it the default for feedback.
- Phase 3 (Months 4-5, growth): Deploy vLLM with Llama 3.1 8B on the same Hetzner GPU used for Whisper (share the GPU during off-peak Whisper hours). Route curriculum generation and batch feedback to self-hosted. Monitor quality weekly.
- Phase 4 (Month 6+, scale): If user base exceeds 50,000, deploy a dedicated GPU instance for LLM inference. Evaluate Llama 3.1 70B for conversation simulation quality. Consider fine-tuning Llama 8B on Auravo's accumulated feedback data (with user consent) to create a specialized coaching model that matches GPT-4o-mini quality at zero API cost.

Technical Needs

Speech-to-Text Engine: Use OpenAI Whisper (self-hosted via faster-whisper) for high-accuracy real-time transcription supporting multiple English accents (Indian, American, British). For short, on-device inference use Whisper.cpp to reduce server load. Must handle background noise gracefully.

NLP and AI Analysis Layer: Use GPT-4o-mini for conversation simulation and feedback generation with structured JSON output. Use Librosa for audio feature extraction (pitch, rate, pauses, energy) and spaCy/LanguageTool for grammar and sentence-structure checks. Route high-volume feedback calls through Groq or self-hosted Llama 3.1 8B via the hybrid routing architecture.

Conversational AI Engine: Integrate GPT-4o-mini for multi-turn dialogue with session memory and in-character simulation. Provide fallback to self-hosted Llama 3.1 8B (via Ollama or vLLM) and Groq API where cost requires. Use the LLMRouter class to manage provider selection and automatic failover.

Audio Recording and Storage: Client-side recording via Web Audio API/MediaRecorder with secure upload to Cloudflare R2 (primary) and AWS S3 as fallback. Store compressed Opus files for cost-effective storage and playback.

Frontend: React.js + Next.js for web, React Native for cross-platform mobile, Tailwind CSS for UI styling, and edge-hosted frontends on Vercel with Cloudflare CDN for performance.

Backend API: Node.js with Express.js or Fastify. Use PostgreSQL for relational data, Redis for caching and real-time features, and Bull/BullMQ for background jobs (audio processing, batch Whisper runs, LLM feedback generation, report generation).

Curriculum Engine: Rule-based and ML-driven system implemented as backend services that map assessment scores to learning content, adapt after each session using stored metrics, and manage difficulty progression. Weekly curriculum plans generated via GPT-4o-mini or self-hosted Llama 3.1 8B with cached baseline templates for each level-goal combination.

Integration Points

- Speech-to-text: OpenAI Whisper (self-hosted via faster-whisper) and Whisper API (fallback)
- LLM Primary: OpenAI GPT-4o-mini (conversation simulation, meeting prep)
- LLM Secondary: Groq API with Llama 3.1 8B (session feedback, curriculum generation)
- LLM Tertiary: Self-hosted vLLM with Llama 3.1 8B/70B (batch processing, cost optimization)
- Payments: Stripe and Razorpay
- Emails: Resend
- Analytics: PostHog
- Push notifications: Firebase Cloud Messaging

Data Storage and Privacy

User audio recordings are encrypted at rest (AES-256) and in transit (TLS 1.3). Recordings are stored in cloud object storage — Cloudflare R2 (primary) with AWS S3 as fallback — with per-user access controls. Users can delete their recordings and account data at any time (GDPR and data privacy compliance). Transcripts and AI feedback are stored in a relational database linked to user accounts. No audio data is shared with third parties for purposes other than transcription processing. Transcription APIs are configured to not retain audio. When using self-hosted Whisper and LLM models, all data processing occurs on Auravo-controlled infrastructure with no third-party data exposure. Privacy policy clearly communicates what data is collected, how it is used, and user rights.

Scalability and Performance

Target architecture supports 10,000 concurrent users at launch, scalable to 100,000 within 12 months. Audio processing and AI inference are the primary cost drivers. Use asynchronous processing queues, read replicas, and edge caching to manage load and optimize cost. CDN for static assets and pre-built scenario content. Rate limiting on AI API calls per user to manage cost and prevent abuse. The hybrid LLM routing architecture automatically shifts load from paid APIs to self-hosted models as GPU capacity increases. Hetzner Cloud GPU instances and Cloudflare R2 zero-egress storage significantly reduce scaling costs compared to an AWS-only architecture.

Potential Challenges

- Speech recognition accuracy for non-native speakers with heavy accents. Mitigation: start with Whisper (self-hosted) and fine-tune or add accent-specific pre-processing with user consent.
- AI feedback quality and consistency. Mitigation: build a human-reviewed rubric for scoring, run periodic LLM-as-judge audits on AI-generated feedback, maintain a golden set of 100 reference pairs, and allow users to flag unhelpful feedback.
- Audio storage costs scaling with user base. Mitigation: compress audio aggressively, auto-delete free-tier recordings after 12 months, and offer extended storage as a paid feature.
- Latency in AI response during simulated conversations. Mitigation: use streaming responses, GPT-4o-mini for low-latency inference, and pre-generate likely follow-ups to reduce perceived wait time.
- User drop-off during onboarding assessment. Mitigation: keep the assessment short (under 7 minutes), allow pausing and resuming, and show a progress bar.
- Managing self-hosted model infrastructure (Whisper, Llama/Mistral via vLLM). Mitigation: start with API-based inference for simplicity, migrate to self-hosted only when volume justifies the operational overhead. Use containerized deployments (Docker) for reproducible GPU environments. Share GPU resources between Whisper and LLM inference during off-peak hours.
- LLM provider lock-in risk. Mitigation: the hybrid routing architecture and OpenAI-compatible API standard (used by vLLM, Groq, and OpenAI) ensures Auravo can switch providers with a config change, not a code change.
- Prompt quality degradation over time as models update. Mitigation: prompt versioning in the database, automated golden set regression testing before deploying new prompt versions, and weekly quality monitoring via LLM-as-judge.

Milestones and Sequencing

Project Estimate

Medium: 6–8 weeks from kickoff to public MVP launch.

Team Size and Composition

Small Team: 4 people total.

- 1 Product Manager / Designer (owns requirements, designs screens, runs user testing)
- 2 Full-Stack Engineers (one leaning backend/AI integration, one leaning frontend/mobile)
- 1 AI/ML Engineer (owns speech analysis pipeline, LLM prompt engineering, curriculum engine, and conversation AI tuning)

Suggested Phases

Phase 1: Foundation and Assessment (Weeks 1–2)

- Key Deliverables:

    - Product Manager: Finalize UI wireframes for onboarding, assessment, and dashboard. Set up analytics tracking plan.
    - Engineers: Set up project infrastructure (repo, CI/CD, cloud environment). Build authentication and user account system. Integrate Whisper API for initial transcription. Build the initial assessment flow (recording, processing, scoring).
    - AI/ML Engineer: Define scoring rubrics for six skill dimensions. Build the baseline scoring pipeline from audio input. Set up GPT-4o-mini integration with initial prompt templates for feedback generation.
- Dependencies: OpenAI API key provisioned. Cloud infrastructure on Railway and Vercel deployed.

Phase 2: Adaptive Learning and Simulations (Weeks 3–5)

- Key Deliverables:
    - Product Manager: Design and test simulation UI and daily session flow. Source and write 30 scenario scripts.
    - Engineers: Build daily session flow (exercise delivery, recording, feedback display). Build simulation conversation interface with turn-by-turn audio interaction. Implement curriculum engine that generates and adapts weekly plans. Deploy self-hosted faster-whisper on Hetzner GPU.
    - AI/ML Engineer: Build and test conversation simulation prompts across all difficulty levels. Tune feedback generation prompts for grammar, filler words, vocabulary, pacing, and tone. Build adaptation logic that re-weights learning plans after each session. Implement prompt versioning system.
- Dependencies: Scoring pipeline from Phase 1 must be functional. Conversation simulation prompts validated against golden set.

Phase 3: Progress Tracking and Meeting Prep (Weeks 5–7)

- Key Deliverables:
    - Product Manager: Design progress dashboard, timeline view, side-by-side replay, and Meeting Prep UI. Conduct usability testing with 5–10 beta users.
    - Engineers: Build progress timeline, charts, milestone detection, and side-by-side replay player. Build Meeting Prep flow (agenda input, context config, simulation, feedback). Implement PDF export for progress reports. Integrate payment gateway and subscription plans. Build LLMRouter with fallback chain.
    - AI/ML Engineer: Build meeting prep prompts (talking points + simulation). Tune milestone detection thresholds. Set up Groq API integration for feedback routing. Begin response caching implementation.
- Dependencies: Session recording storage from Phase 2. Scoring data accumulated from test usage. Groq API account provisioned.

Phase 4: Polish, Testing, and Launch (Weeks 7–8)

- Key Deliverables:
    - Product Manager: Final QA pass, app store listing copy, launch communications, and onboarding email sequences.

    - Engineers: Performance optimization, bug fixes, load testing for 10,000 concurrent users. Deploy to production. Set up monitoring and alerting (Prometheus, Grafana, Sentry).
    - AI/ML Engineer: Audit AI feedback quality across 50+ test sessions using LLM-as-judge. Adjust scoring calibration based on beta feedback. Validate hybrid routing between GPT-4o-mini and Groq. Finalize prompt versions for launch.
- Dependencies: Beta user feedback incorporated. Payment system tested end-to-end. LLM quality metrics meet thresholds (golden set 90%+ match, thumbs-down rate below 15%).